23520825 Vo Anh Kiet Lab04

Hệ Điều Hành (Đại Học Quốc Gia Thành Phố Hồ Chí Minh Trường Đại Hoc Công Nghê Thông Tin)

Báo cáo thực hành môn Hệ điều hành - Giảng viên: Thân Thế Tùng.

Họ và tên: Võ Anh Kiệt
Mã số sinh viên: 23520825
Lớp: IT007.P14.1

HỆ ĐIỀU HÀNH
BÁO CÁO LAB 4

CHECKLIST

3.5. BÀI TẬP THỰC HÀNH

	BT 1	BT 2
Vẽ lưu đồ giải thuật	☒	☒
Chạy tay lưu đồ giải thuật	☐	☐
Hiện thực code	☒	☒
Chạy code và kiểm chứng	☒	☒

3.6. BÀI TẬP ÔN TẬP

	BT 1
Vẽ lưu đồ giải thuật	☒
Chạy tay lưu đồ giải thuật	☐
Hiện thực code	☒
Chạy code và kiểm chứng	☒

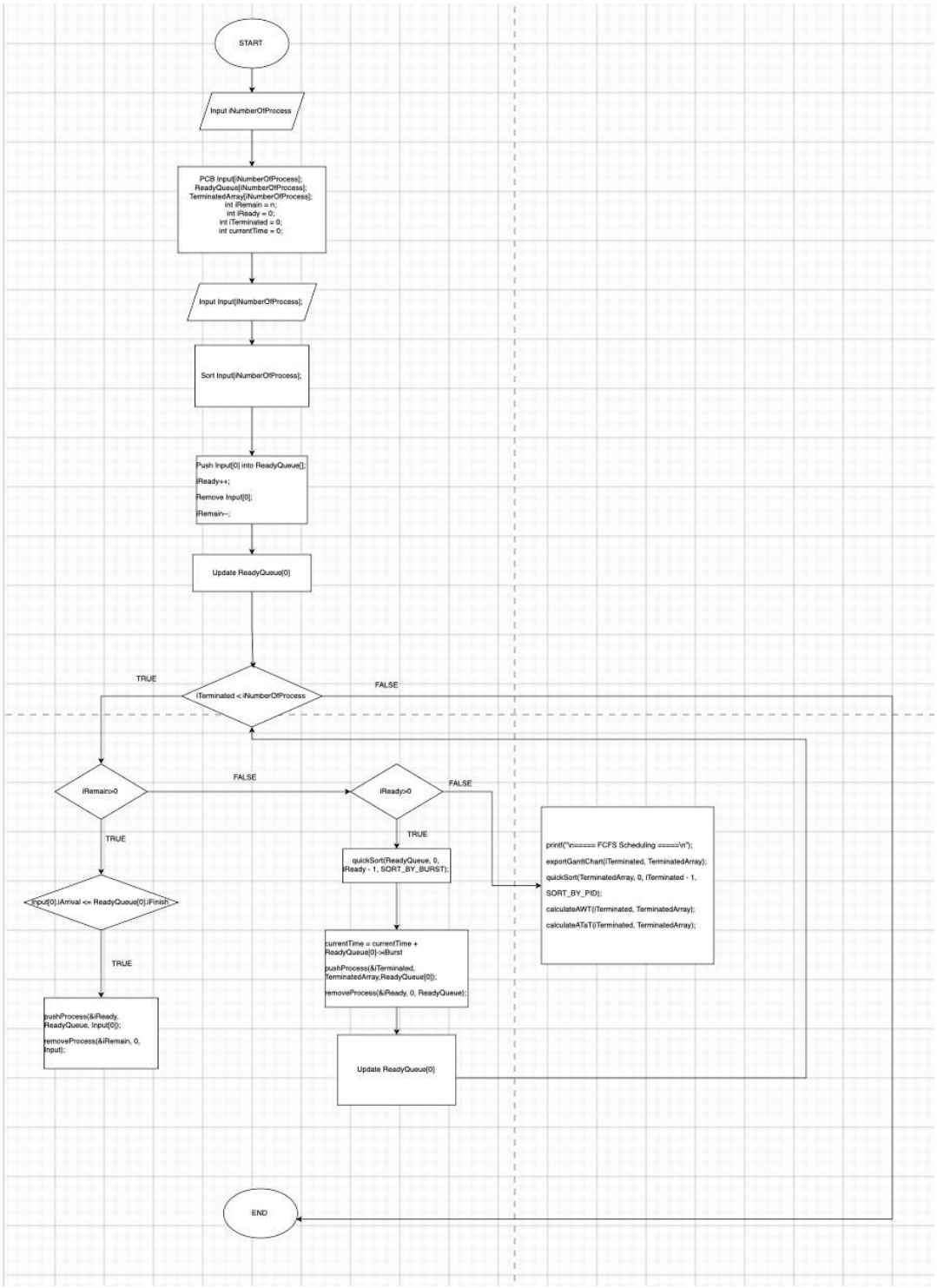
Tự chấm điểm: 9

**Lưu ý: Xuất báo cáo theo định dạng PDF, đặt tên theo cú pháp:*
<Tên nhóm>_LAB3.pdf

2.5. BÀI TẬP THỰC HÀNH

1. Giải thuật Shortest-Job-First

Trả lời...



```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>

#define SORT_BY_ARRIVAL 0
#define SORT_BY_PID 1
#define SORT_BY_BURST 2
#define SORT_BY_START 3

typedef struct
{
    int iPID;
    int iArrival, iBurst;
    int iStart, iFinish, iWaiting, iResponse, iTaT;
    int remainingTime;
} PCB;

int partition(PCB P[], int low, int high, int iCriteria);
int swapProcess(PCB *P, PCB *Q);

void inputProcess(int n, PCB P[])
{
    for (int i = 0; i < n; i++)
    {
        P[i].iPID = i + 1;
        P[i].iArrival = rand() % 21; // Tạo Arrival Time ngẫu nhiên trong đoạn [0, 20]
        P[i].iBurst = (rand() % 11) + 2; // Tạo Burst Time ngẫu nhiên trong đoạn [2, 12]
        P[i].iStart = P[i].iFinish = P[i].iWaiting = P[i].iResponse = P[i].iTaT = 0;
        P[i].remainingTime = P[i].iBurst;
    }
}

void printProcess(int n, PCB P[])
{
```

```
    printf("PID\tArrival\tBurst\tStart\tFinish\tWaiting\tResponse\tTaT\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t\t%d\n",
                P[i].iPID, P[i].iArrival, P[i].iBurst, P[i].iStart, P[i].iFinish,
                P[i].iWaiting, P[i].iResponse, P[i].iTaT);
    }
}

void exportGanttChart(int n, PCB P[])
{
    printf("Gantt Chart:\n|");
    for (int i = 0; i < n; i++)
    {
        printf(" P%d |", P[i].iPID);
    }
    printf("\n0");
    for (int i = 0; i < n; i++)
    {
        printf(" %d", P[i].iFinish);
    }
    printf("\n");
}

void calculateAWT(int n, PCB P[])
{
    int totalWaiting = 0;
    for (int i = 0; i < n; i++)
    {
        totalWaiting += P[i].iWaiting;
    }
    float averageWaiting = (float)totalWaiting / n;
    printf("Average Waiting Time: %.2f\n", averageWaiting);
}

void calculateATaT(int n, PCB P[])
```

```
{
    int totalTaT = 0;
    for (int i = 0; i < n; i++)
    {
        totalTaT += P[i].iTaT;
    }
    float averageTaT = (float)totalTaT / n;
    printf("Average Turnaround Time: %.2f\n", averageTaT);
}

void quickSort(PCB P[], int low, int high, int iCriteria)
{
    if (low < high)
    {
        int pi = partition(P, low, high, iCriteria);
        quickSort(P, low, pi - 1, iCriteria);
        quickSort(P, pi + 1, high, iCriteria);
    }
}

int partition(PCB P[], int low, int high, int iCriteria)
{
    PCB pivot = P[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        int condition = 0;
        if (iCriteria == SORT_BY_ARRIVAL)
            condition = (P[j].iArrival < pivot.iArrival);
        else if (iCriteria == SORT_BY_PID)
            condition = (P[j].iPID < pivot.iPID);
        else if (iCriteria == SORT_BY_BURST)
            condition = (P[j].iBurst < pivot.iBurst);
        else if (iCriteria == SORT_BY_START)
            condition = (P[j].iStart < pivot.iStart);
```

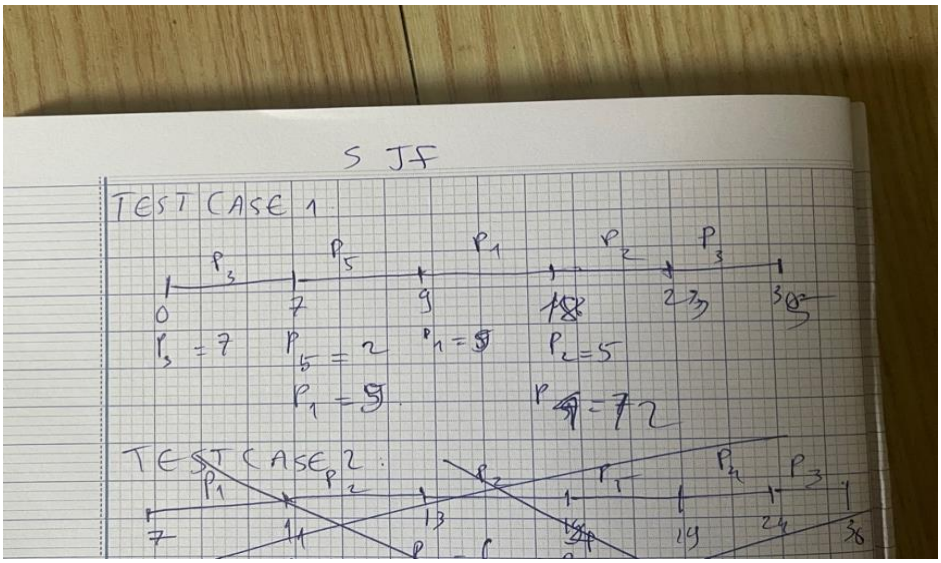
```
        if (condition)
        {
            i++;
            swapProcess(&P[i], &P[j]);
        }
    }
    swapProcess(&P[i + 1], &P[high]);
    return i + 1;
}

int swapProcess(PCB *P, PCB *Q)
{
    PCB temp = *P;
    *P = *Q;
    *Q = temp;
    return 1;
}

void pushProcess(int *n, PCB P[], PCB process)
{
    P[*n] = process;
    (*n)++;
}

void removeProcess(int *n, int index, PCB P[])
{
    for (int i = index; i < *n - 1; i++)
    {
        P[i] = P[i + 1];
    }
    (*n)--;
}

int main()
{
    srand(time(NULL));
```

```
PCB Input[10];
PCB ReadyQueue[10];
PCB TerminatedArray[10];
int iNumberOfProcess;
printf("Please input number of Process: ");
scanf("%d", &iNumberOfProcess);

int iRemain = iNumberOfProcess, iTerminated = 0, iReady = 0;
inputProcess(iNumberOfProcess, Input);
quickSort(Input, 0, iNumberOfProcess - 1, SORT_BY_ARRIVAL);
int currentTime = 0;
while (iTerminated < iNumberOfProcess)
{
   // Move processes from Input to ReadyQueue if they have arrived
   for (int i = 0; i < iRemain; i++)
   {
      if (Input[i].iArrival <= currentTime)
      {
         pushProcess(&iReady, ReadyQueue, Input[i]);
         removeProcess(&iRemain, i, Input);
         i--; // Adjust index after removal
      }
   }

   // Sort ReadyQueue by Burst Time to always pick the shortest job first
   quickSort(ReadyQueue, 0, iReady - 1, SORT_BY_BURST);

   if (iReady == 0)
   {
      currentTime++;
      continue;
   }

   PCB *currentProcess = &ReadyQueue[0];

   if (currentTime < currentProcess->iArrival)
```

```
        {
            currentTime = currentProcess->iArrival;
        }

        currentProcess->iStart = currentTime;
        currentProcess->iResponse = currentProcess->iStart - currentProcess->iArrival;
        currentTime += currentProcess->remainingTime;
        currentProcess->iFinish = currentTime;
        currentProcess->iTaT = currentProcess->iFinish - currentProcess->iArrival;
        currentProcess->iWaiting = currentProcess->iTaT - currentProcess->iBurst;

        pushProcess(&iTerminated, TerminatedArray, *currentProcess);
        removeProcess(&iReady, 0, ReadyQueue);
    }
    printf("\n===== SJF Scheduling =====\n");
    exportGanttChart(iTerminated, TerminatedArray);
    quickSort(TerminatedArray, 0, iTerminated - 1, SORT_BY_PID);
    printProcess(iTerminated, TerminatedArray);
    calculateAWT(iTerminated, TerminatedArray);
    calculateATaT(iTerminated, TerminatedArray);
    return 0;
}
```

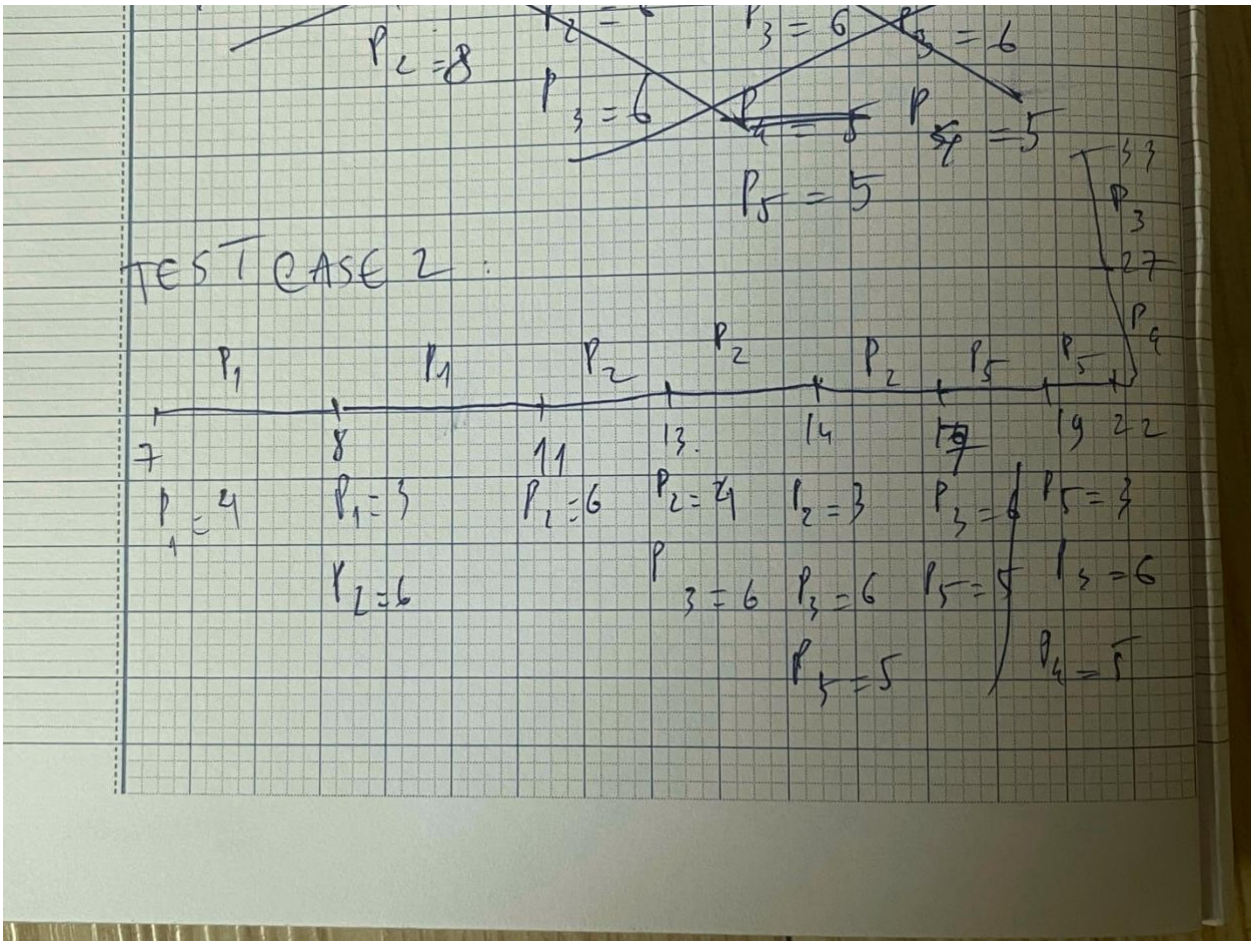
TEST CASE 1:

```
PROBLEMS 5   OUTPUT   TERMINAL   DEBUG CONSOLE   PORTS
ubuntu@VoAnhKiet-23520825:~$ gcc ./Lab04/SJF.c -o SJF
ubuntu@VoAnhKiet-23520825:~$ ./SJF
Please input number of Process: 5

===== SJF Scheduling =====
Gantt Chart:
| P3 | P5 | P1 | P2 | P4 |
0 7 9 18 23 35
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       5       9       9       18      4       4               13
2       11      5       18      23      7       7               12
3       0       7       0       7       0       0               7
4       17      12      23      35      6       6               18
5       4       2       7       9       3       3               5
Average Waiting Time: 4.00
Average Turnaround Time: 11.00
ubuntu@VoAnhKiet-23520825:~$
```



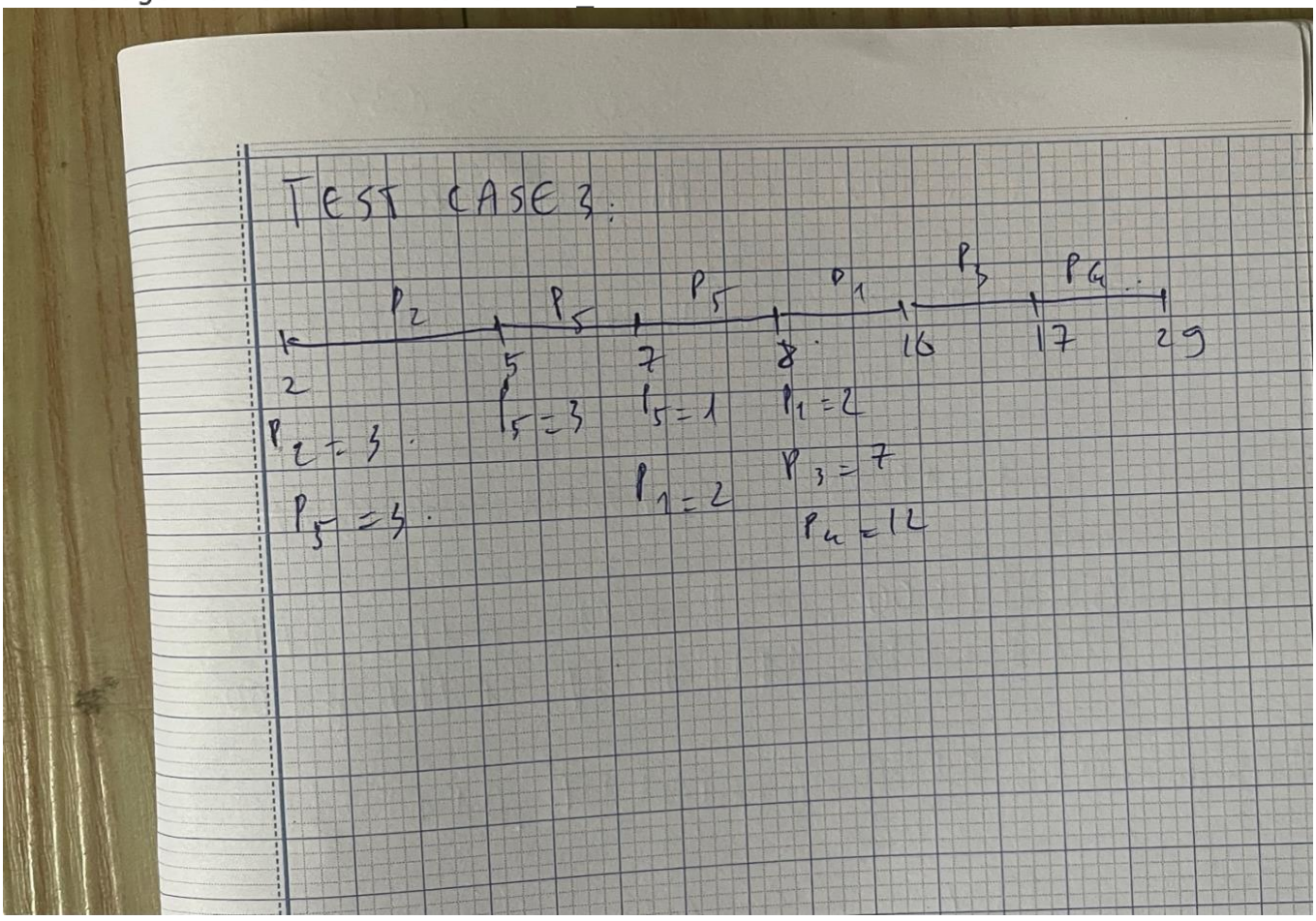
TEST CASE 2:

```
ubuntu@VoAnhKiet-23520825:~$ ./SJF
Please input number of Process: 5

===== SJF Scheduling =====
Gantt Chart:
| P1 | P2 | P5 | P4 | P3 |
0 11 17 22 27 33
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       7       4       7       11      0       0               4
2       8       6       11      17      3       3               9
3       13      6       27      33      14      14              20
4       19      5       22      27      3       3               8
5       14      5       17      22      3       3               8
Average Waiting Time: 4.60
Average Turnaround Time: 9.80
```



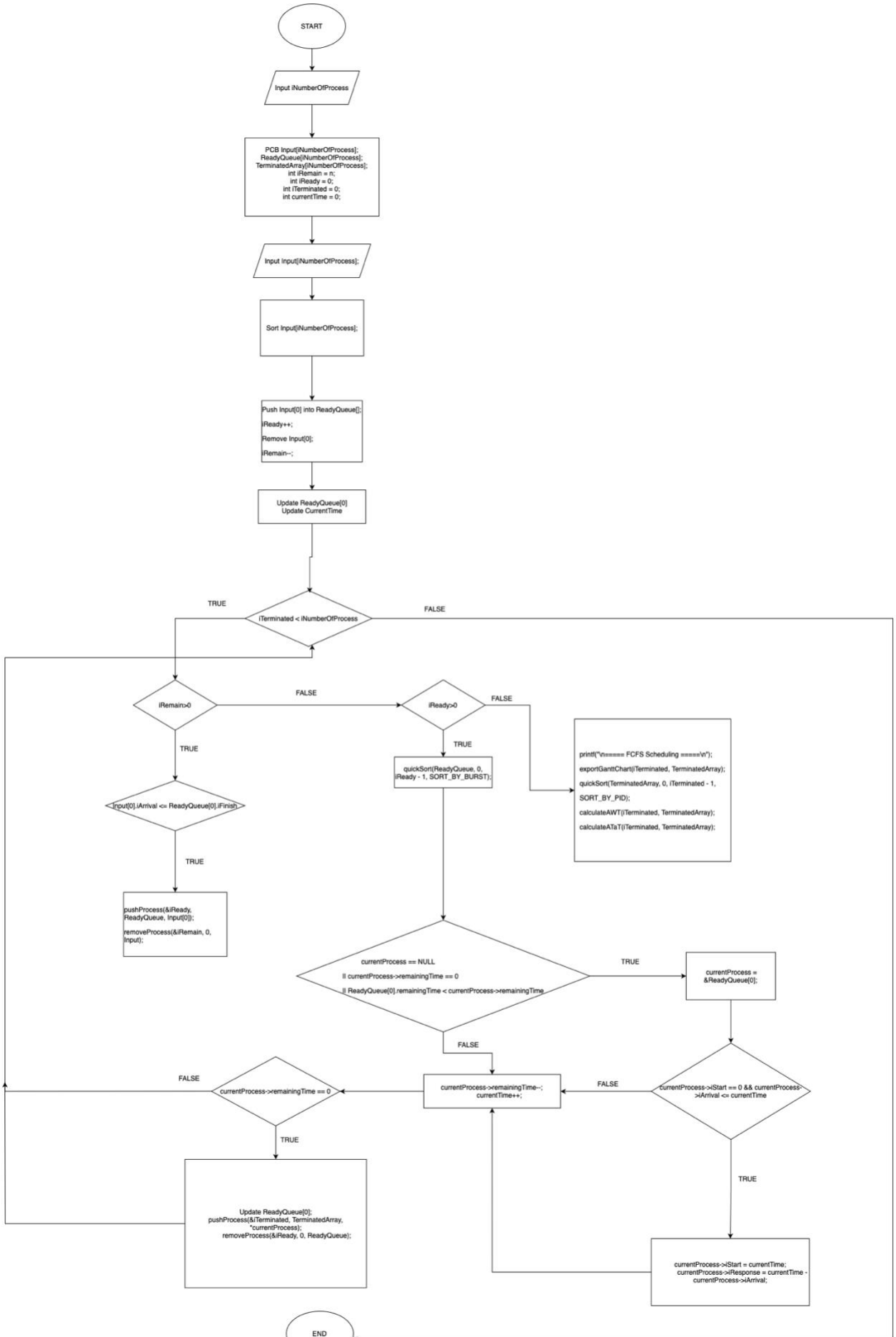
TEST CASE 3:

```
ubuntu@VoAnhKiet-23520825:~$ ./SJF
Please input number of Process: 5

===== SJF Scheduling =====
Gantt Chart:
| P2 | P5 | P1 | P3 | P4 |
0 5 8 10 17 29
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       7       2       8       10      1       1               3
2       2       3       2       5       0       0               3
3       8       7       10      17      2       2               9
4       8       12      17      29      9       9               21
5       2       3       5       8       3       3               6
Average Waiting Time: 3.00
Average Turnaround Time: 8.40
```



2. Giải thuật Shortest-Remaining-Time-First hoặc Round Robin

Trả lời...



```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>
#define SORT_BY_ARRIVAL 0
#define SORT_BY_PID 1
#define SORT_BY_BURST 2
#define SORT_BY_START 3

typedef struct
{
    int iPID;
    int iArrival, iBurst;
    int iStart, iFinish, iWaiting, iResponse, iTaT;
    int remainingTime;
} PCB;

int partition(PCB P[], int low, int high, int iCriteria);
int swapProcess(PCB *P, PCB *Q);

void inputProcess(int n, PCB P[])
{
    for (int i = 0; i < n; i++)
    {
        P[i].iPID = i + 1;
        P[i].iArrival = rand() % 21; // Tạo arrival Time ngẫu nhiên trong đoạn [0, 20]
        P[i].iBurst = (rand() % 11) + 2; // Tạo Burst Time ngẫu nhiên trong đoạn [2, 12]
        P[i].iStart = P[i].iFinish = P[i].iWaiting = P[i].iResponse = P[i].iTaT = 0;
        P[i].remainingTime = P[i].iBurst;
    }
}

void printProcess(int n, PCB P[])
{
    printf("PID\tArrival\tBurst\tStart\tFinish\tWaiting\tResponse\tTaT\n");
    for (int i = 0; i < n; i++)
```

```
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t\t%d\n",
                P[i].iPID, P[i].iArrival, P[i].iBurst, P[i].iStart, P[i].iFinish,
                P[i].iWaiting, P[i].iResponse, P[i].iTaT);
    }
}

void exportGanttChart(int totalTime, int ganttChart[], int n)
{
    printf("Gantt Chart:\n|");
    int prevPID = -1;
    for (int i = 0; i < totalTime; i++)
    {
        if (ganttChart[i] != prevPID && ganttChart[i] != -1)
        {
            if (prevPID != -1)
                printf(" |");
            printf(" P%d", ganttChart[i]);
            prevPID = ganttChart[i];
        }
    }

    printf(" |\n");
    // In dòng thời gian phía dưới
    prevPID = ganttChart[0];
    printf("0");
    for (int i = 1; i <= totalTime; i++)
    {
        if (i == totalTime || ganttChart[i] != prevPID)
        {
            if (prevPID != -1 || i == totalTime)
                printf("  %2d ", i);
            prevPID = ganttChart[i];
        }
    }
    printf("\n");
```

```
}

void calculateAWT(int n, PCB P[])
{
    int totalWaiting = 0;
    for (int i = 0; i < n; i++)
        totalWaiting += P[i].iWaiting;
    float averageWaiting = (float)totalWaiting / n;
    printf("Average Waiting Time: %.2f\n", averageWaiting);
}

void calculateTAT(int n, PCB P[])
{
    int totalTaT = 0;
    for (int i = 0; i < n; i++)
        totalTaT += P[i].iTaT;
    float averageTaT = (float)totalTaT / n;
    printf("Average Turnaround Time: %.2f\n", averageTaT);
}

void quickSort(PCB P[], int low, int high, int iCriteria)
{
    if (low < high)
    {
        int pi = partition(P, low, high, iCriteria);
        quickSort(P, low, pi - 1, iCriteria);
        quickSort(P, pi + 1, high, iCriteria);
    }
}

int partition(PCB P[], int low, int high, int iCriteria)
{
    PCB pivot = P[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
```

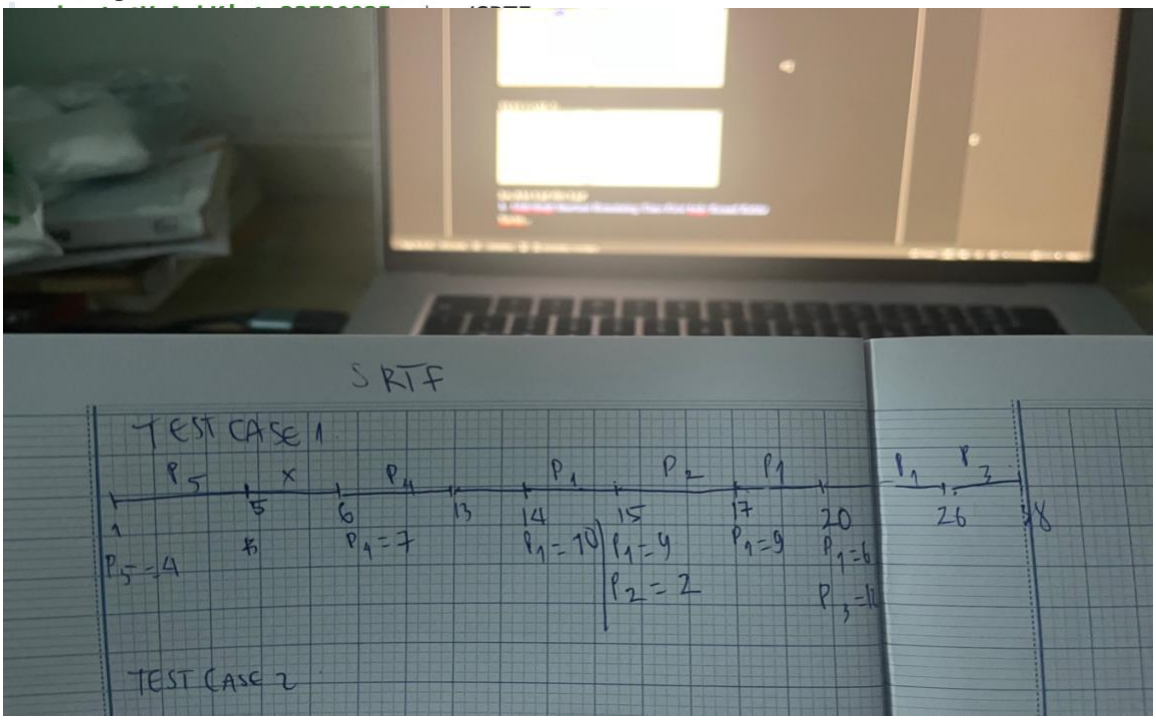
```
        int condition = 0;
        if (iCriteria == SORT_BY_ARRIVAL)
            condition = (P[j].iArrival < pivot.iArrival);
        else if (iCriteria == SORT_BY_PID)
            condition = (P[j].iPID < pivot.iPID);
        else if (iCriteria == SORT_BY_BURST)
            condition = (P[j].iBurst < pivot.iBurst);
        else if (iCriteria == SORT_BY_START)
            condition = (P[j].iStart < pivot.iStart);

        if (condition)
        {
            i++;
            swapProcess(&P[i], &P[j]);
        }
    }
    swapProcess(&P[i + 1], &P[high]);
    return i + 1;
}

int swapProcess(PCB *P, PCB *Q)
{
    PCB temp = *P;
    *P = *Q;
    *Q = temp;
    return 1;
}

void pushProcess(int *n, PCB P[], PCB process)
{
    P[*n] = process;
    (*n)++;
}

void removeProcess(int *n, int index, PCB P[])
{
```

```
    for (int i = index; i < *n - 1; i++)
        P[i] = P[i + 1];
    (*n)--;
}

int findShortestJob(int n, PCB P[], int currentTime)
{
    int index = -1;
    int minBurst = INT_MAX;
    for (int i = 0; i < n; i++)
    {
        if (P[i].iArrival <= currentTime && P[i].remainingTime > 0 && P[i].remainingTime < minBurst)
        {
            minBurst = P[i].remainingTime;
            index = i;
        }
    }
    return index;
}

int main()
{
    srand(time(NULL));
    PCB Input[10];
    PCB TerminateArray[10];
    int ganttChart[100] = {-1};
    int iNumberOfProcess;
    printf("Please input number of Process: ");
    scanf("%d", &iNumberOfProcess);
    int iRemain = iNumberOfProcess;
    int iTerminated = 0;

    inputProcess(iNumberOfProcess, Input);
    quickSort(Input, 0, iNumberOfProcess - 1, SORT_BY_ARRIVAL);

    int currentTime = 0;
```

```
    while (iTerminated < iNumberOfProcess)
    {
        int idx = findShortestJob(iRemain, Input, currentTime);
        if (idx == -1)
        {
            ganttChart[currentTime] = -1;
            currentTime++;
            continue;
        }
        PCB *currentProcess = &Input[idx];
        if (currentProcess->remainingTime == currentProcess->iBurst)
        {
            currentProcess->iStart = currentTime;
            currentProcess->iResponse = currentProcess->iStart - currentProcess->iArrival;
        }
        currentProcess->remainingTime--;
        ganttChart[currentTime] = currentProcess->iPID;
        currentTime++;
        if (currentProcess->remainingTime == 0)
        {
            currentProcess->iFinish = currentTime;
            currentProcess->iTaT = currentProcess->iFinish - currentProcess->iArrival;
            currentProcess->iWaiting = currentProcess->iTaT - currentProcess->iBurst;
            pushProcess(&iTerminated, TerminateArray, *currentProcess);
            removeProcess(&iRemain, idx, Input);
        }
    }
    printf("\n==== SRTF Scheduling =====\n");
    exportGanttChart(currentTime, ganttChart, currentTime);
    quickSort(TerminateArray, 0, iTerminated - 1, SORT_BY_PID);
    printProcess(iTerminated, TerminateArray);
    calculateAWT(iTerminated, TerminateArray);
    calculateTAT(iTerminated, TerminateArray);
    return 0;
}
```

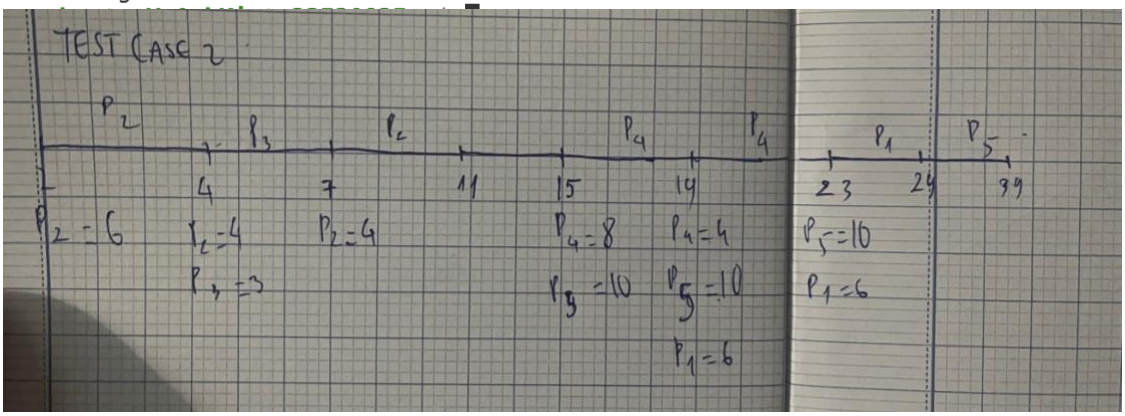
TEST CASE 1:

```
ubuntu@VoAnhKiet-23520825:~$ gcc ./Lab04/SRTF.c -o SRTF
ubuntu@VoAnhKiet-23520825:~$ ./SRTF
Please input number of Process: 5

==== SRTF Scheduling =====
Gantt Chart:
| P5 | P4 | P1 | P2 | P1 | P3 |
0   5   13   15   17   26   38
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       14      10      14      26      2       0               12
2       15      2       15      17      0       0               2
3       20      12      26      38      6       6               18
4       6       7       6       13      0       0               7
5       1       4       1       5       0       0               4
Average Waiting Time: 1.60
Average Turnaround Time: 8.60
```



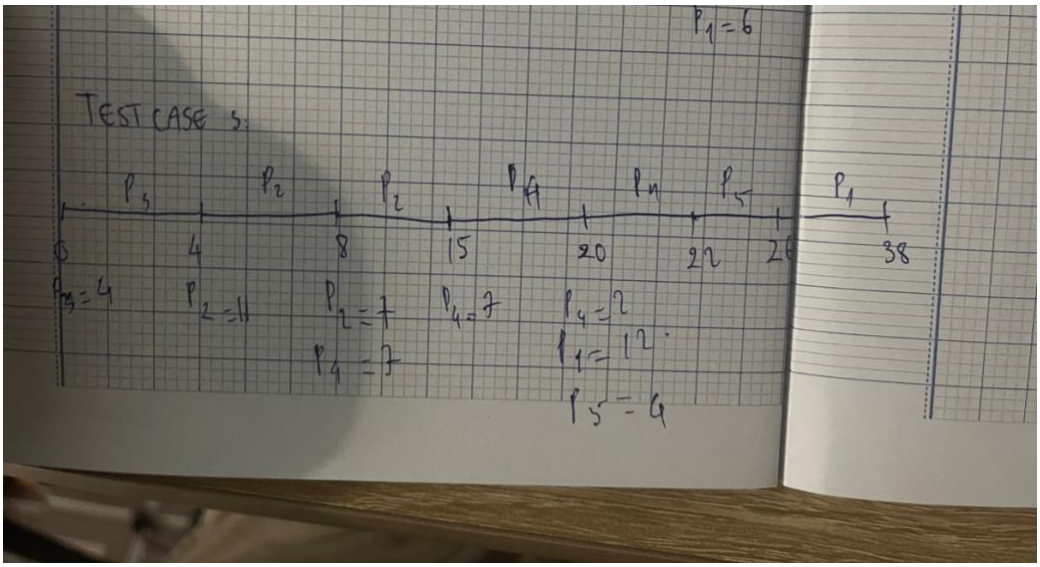
TEST CASE 2:

```
ubuntu@VoAnhKiet-23520825:~$ ./SRTF
Please input number of Process: 5

==== SRTF Scheduling =====
Gantt Chart:
| P2 | P3 | P2 | P4 | P1 | P5 |
0   4    7   11   23   29   39
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       19      6       23      29      4       4               10
2       2       6       2       11      3       0               9
3       4       3       4       7       0       0               3
4       15      8       15      23      0       0               8
5       15      10      29      39      14      14              24
Average Waiting Time: 4.20
Average Turnaround Time: 10.80
```



TEST CASE 3:

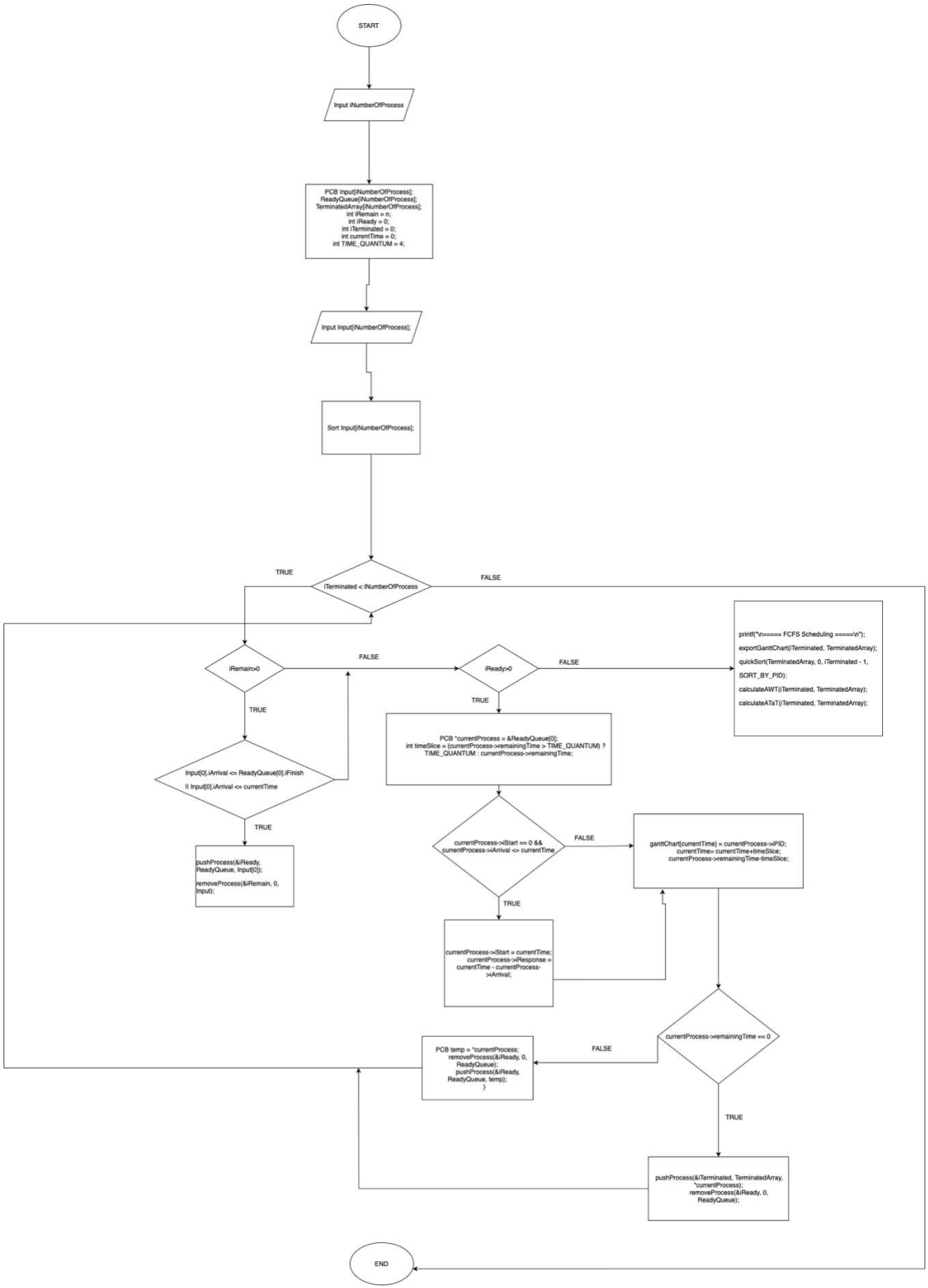
```
ubuntu@VoAnhKiet-23520825:~$ ./SRTF
Please input number of Process: 5

==== SRTF Scheduling =====
Gantt Chart:
| P3 | P2 | P4 | P5 | P1 |
0   4   15   22   26   38
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       20      12      26      38      6       6               18
2       4       11      4       15      0       0               11
3       0       4       0       4       0       0               4
4       8       7       15      22      7       7               14
5       20      4       22      26      2       2               6
Average Waiting Time: 3.00
Average Turnaround Time: 10.
```



2.6. BÀI TẬP ÔN TẬP

1. Giải thuật Shortest-Remaining-Time-First hoặc Round Robin

Trả lời...



```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>
#define SORT_BY_ARRIVAL 0
#define SORT_BY_PID 1
#define SORT_BY_BURST 2
#define SORT_BY_START 3
#define TIME_QUANTUM 4

typedef struct
{
    int iPID;
    int iArrival, iBurst;
    int iStart, iFinish, iWaiting, iResponse, iTaT;
    int remainingTime;
} PCB;

int partition(PCB P[], int low, int high, int iCriteria);
int swapProcess(PCB *P, PCB *Q);

void inputProcess(int n, PCB P[])
{
    for (int i = 0; i < n; i++)
    {
        P[i].iPID = i + 1;
        P[i].iArrival = rand() % 21; // Tạo Arrival Time ngẫu nhiên trong đoạn [0, 20]
        P[i].iBurst = (rand() % 11) + 2; // Tạo Burst Time ngẫu nhiên trong đoạn [2, 12]
        P[i].iStart = P[i].iFinish = P[i].iWaiting = P[i].iResponse = P[i].iTaT = 0;
        P[i].remainingTime = P[i].iBurst;
    }
}

void printProcess(int n, PCB P[])
{
```

```
    printf("PID\tArrival\tBurst\tStart\tFinish\tWaiting\tResponse\tTaT\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t\t%d\n",
                P[i].iPID, P[i].iArrival, P[i].iBurst, P[i].iStart, P[i].iFinish,
                P[i].iWaiting, P[i].iResponse, P[i].iTaT);
    }
}

void exportGanttChart(int n, int ganttChart[], int totalTime)
{
    printf("Gantt Chart:\n|");
    for (int i = 0; i < totalTime; i++)
    {
        if (ganttChart[i] != -1)
        {
            printf(" P%d |", ganttChart[i]);
        }
    }
    printf("\n0");
    for (int i = 0; i < totalTime; i++)
    {
        if (ganttChart[i] != -1)
        {
            printf(" %d", i + 1);
        }
    }
    printf("\n");
}

void calculateAWT(int n, PCB P[])
{
    int totalWaiting = 0;
    for (int i = 0; i < n; i++)
    {
        totalWaiting += P[i].iWaiting;
```

```
    }
    float averageWaiting = (float)totalWaiting / n;
    printf("Average Waiting Time: %.2f\n", averageWaiting);
}

void calculateATaT(int n, PCB P[])
{
    int totalTaT = 0;
    for (int i = 0; i < n; i++)
    {
        totalTaT += P[i].iTaT;
    }
    float averageTaT = (float)totalTaT / n;
    printf("Average Turnaround Time: %.2f\n", averageTaT);
}

void quickSort(PCB P[], int low, int high, int iCriteria)
{
    if (low < high)
    {
        int pi = partition(P, low, high, iCriteria);
        quickSort(P, low, pi - 1, iCriteria);
        quickSort(P, pi + 1, high, iCriteria);
    }
}

int partition(PCB P[], int low, int high, int iCriteria)
{
    PCB pivot = P[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        int condition = 0;
        if (iCriteria == SORT_BY_ARRIVAL)
            condition = (P[j].iArrival < pivot.iArrival);
        else if (iCriteria == SORT_BY_PID)
```

```
            condition = (P[j].iPID < pivot.iPID);
        else if (iCriteria == SORT_BY_BURST)
            condition = (P[j].remainingTime < pivot.remainingTime);
        else if (iCriteria == SORT_BY_START)
            condition = (P[j].iStart < pivot.iStart);

        if (condition)
        {
            i++;
            swapProcess(&P[i], &P[j]);
        }
    }
    swapProcess(&P[i + 1], &P[high]);
    return i + 1;
}

int swapProcess(PCB *P, PCB *Q)
{
    PCB temp = *P;
    *P = *Q;
    *Q = temp;
    return 1;
}

void pushProcess(int *n, PCB P[], PCB process)
{
    P[*n] = process;
    (*n)++;
}

void removeProcess(int *n, int index, PCB P[])
{
    for (int i = index; i < *n - 1; i++)
    {
        P[i] = P[i + 1];
    }
```

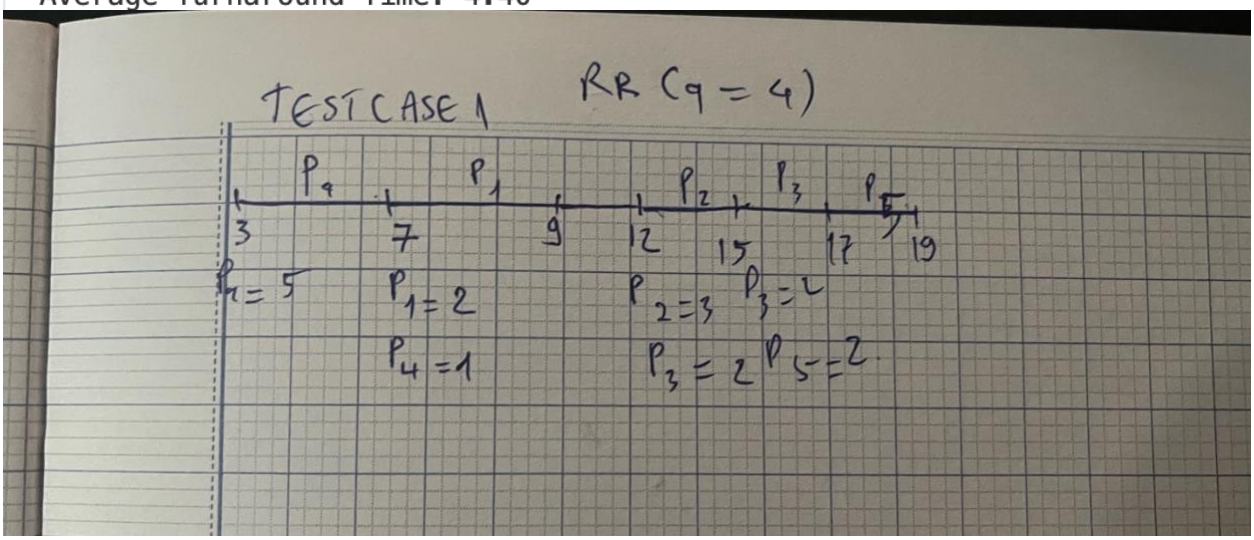
```
    (*n)--;
}

int main()
{
    srand(time(NULL));
    PCB Input[10];
    PCB ReadyQueue[10];
    PCB TerminatedArray[10];
    int ganttChart[100];
    for (int i = 0; i < 100; i++) ganttChart[i] = -1;

    int iNumberOfProcess;
    printf("Please input number of Process: ");
    scanf("%d", &iNumberOfProcess);

    int iRemain = iNumberOfProcess, iTerminated = 0, iReady = 0;
    inputProcess(iNumberOfProcess, Input);
    quickSort(Input, 0, iNumberOfProcess - 1, SORT_BY_ARRIVAL);
    int currentTime = 0;

    while (iTerminated < iNumberOfProcess)
    {
        // Move processes from Input to ReadyQueue if they have arrived
        for (int i = 0; i < iRemain; i++)
        {
            if (Input[i].iArrival <= currentTime)
            {
                pushProcess(&iReady, ReadyQueue, Input[i]);
                removeProcess(&iRemain, i, Input);
                i--; // Adjust index after removal
            }
        }

        if (iReady == 0)
        {
```

```
            ganttChart[currentTime] = -1;
            currentTime++;
            continue;
        }

        PCB *currentProcess = &ReadyQueue[0];
        int timeSlice = (currentProcess->remainingTime > TIME_QUANTUM) ? TIME_QUANTUM : currentProcess-
>remainingTime;

        if (currentProcess->iStart == 0 && currentProcess->iArrival <= currentTime)
        {
            currentProcess->iStart = currentTime;
            currentProcess->iResponse = currentTime - currentProcess->iArrival;
        }

        for (int t = 0; t < timeSlice; t++)
        {
            ganttChart[currentTime] = currentProcess->iPID;
            currentTime++;
            currentProcess->remainingTime--;
        }

        if (currentProcess->remainingTime == 0)
        {
            currentProcess->iFinish = currentTime;
            currentProcess->iTaT = currentProcess->iFinish - currentProcess->iArrival;
            currentProcess->iWaiting = currentProcess->iTaT - currentProcess->iBurst;

            pushProcess(&iTerminated, TerminatedArray, *currentProcess);
            removeProcess(&iReady, 0, ReadyQueue);
        }
        else
        {
            PCB temp = *currentProcess;
            removeProcess(&iReady, 0, ReadyQueue);
            pushProcess(&iReady, ReadyQueue, temp);
```

```
        }
    }
    printf("\n===== Round Robin Scheduling =====\n");
    exportGanttChart(currentTime, ganttChart, currentTime);
    quickSort(TerminatedArray, 0, iTerminated - 1, SORT_BY_PID);
    printProcess(iTerminated, TerminatedArray);
    calculateAWT(iTerminated, TerminatedArray);
    calculateATaT(iTerminated, TerminatedArray);
    return 0;
}
```

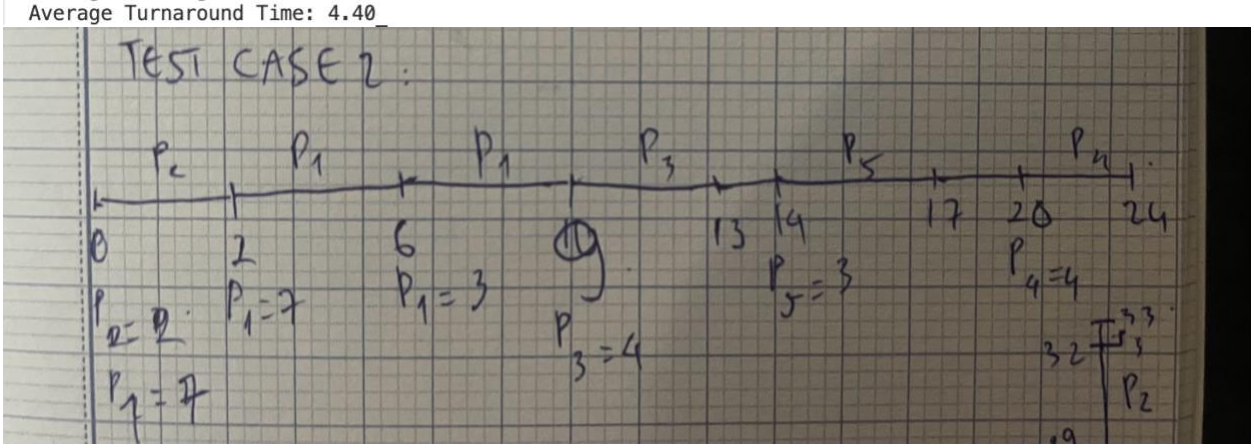
TEST CASE 1:

```
ubuntu@VoAnhKiet-23520825:~$ ./RR
Please input number of Process: 5

===== Round Robin Scheduling =====
Gantt Chart:
| P4 | P4 | P4 | P4 | P4 | P1 | P1 | P2 | P2 | P2 | P3 | P3 | P5 | P5 |
0 4 5 6 7 8 9 10 13 14 15 16 17 18 19
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       6       2       8       10      2       2               4
2       12      3       12      15      0       0               3
3       12      2       15      17      3       3               5
4       3       5       3       8       0       0               5
5       14      2       17      19      3       3               5
Average Waiting Time: 1.60
Average Turnaround Time: 4.40
```



TEST CASE 2:

```
ubuntu@VoAnhKiet-23520825:~$ ./RR
Please input number of Process: 5

===== Round Robin Scheduling =====
Gantt Chart:
| P2 | P2 | P1 | P1 | P1 | P1 | P1 | P1 | P1 | P3 | P3 | P3 | P3 | P5 | P5 | P5 | P4 | P4 | P4 | P4 |
0 1 2 3 4 5 6 7 8 9 10 11 12 13 15 16 17 21 22 23 24
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       0       7       2       9       2       2               9
2       0       2       0       2       0       0               2
3       9       4       9       13      0       0               4
4       20      4       20      24      0       0               4
5       14      3       14      17      0       0               3
Average Waiting Time: 0.40
Average Turnaround Time: 4.40
```



TEST CASE 3:

```
ubuntu@VoAnhKiet-23520825:~$ ./RR
Please input number of Process: 5

===== Round Robin Scheduling =====
Gantt Chart:
| P1 | P1 | P1 | P5 | P5 | P5 | P5 | P2 | P2 | P2 | P2 | P5 | P5 | P5 | P5 | P2 | P2 | P2 | P2 | P4 | P4 | P4 | P4 | P3 | P3 | P3 | P3 | P2 | P2 | P2 | P4 | P4 | P3 |
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
PID     Arrival Burst   Start   Finish  Waiting Response        TaT
1       0       3       0       3       0       0               3
2       2       11      7       30      17      5               28
3       13      5       23      33      15      10              20
4       11      6       19      32      15      8               21
5       2       8       3       15      5       1               13
Average Waiting Time: 10.40
Average Turnaround Time: 17.00
```